03 / 24 — Phase 3 Assessment: State and Hooks

> **Scope.** Lessons 01 through 23 of Phase 3, with prerequisite knowledge from Phases 0–2 assumed. **Format.** Five parts: (A) conceptual explanation, (B) code prediction, (C) debugging challenges, (D) implementation exercises, (E) architecture and reasoning. A detailed answer key and scoring rubric follow. **Time.** 4–5 hours, closed notes for Parts A–C and E; Part D may be done in an editor with TypeScript and the React type definitions available (no AI assistance, no copying from lessons). **Rules.** Write answers *before* reading the key. For every prediction, write the output and one sentence of reasoning. For every debugging challenge, name the root cause, the fix, and how you would verify it. Version baseline: React 19.x (see ROADMAP §0); the predicted outputs in Part B were executed against React 19.3.1 on 2026-10-22. `useEffectEvent` is stable since React 19.2; answers that note "verify against current docs" for fast-moving APIs are still acceptable and rewarded.

How the assessment is scored (summary)

Category	Parts	Points
Explanation	A (12 questions × 3)	36
Reasoning	B (8 × 3) + E (4 × 4)	40
Debugging	C (5 × 6)	30
Implementation	D (3 × 10)	30
Total		**136**

Pass: ≥ 80% overall (≥ 109 / 136) **and** ≥ 70% in *each* category (Explanation ≥ 26, Reasoning ≥ 28, Debugging ≥ 21, Implementation ≥ 21). Full rubric and promotion criteria are in §Scoring and Promotion at the end.

Part A — Conceptual explanation (36 points)

Answer each in 4–8 sentences (or a short list). Each question is worth 3 points: one point for each key idea in the answer key.

A1. (Lessons 01–02) Explain "state as a snapshot". Why does `setCount(count + 1)` called three times in one handler increase the count by 1, and what changes with `setCount(c => c + 1)`? Where is the update stored between the call and the next render?

A2. (Lesson 03) When would you choose `useReducer` over `useState`? Give two technical reasons (not "it's cleaner") and one situation where `useState` is the better tool.

A3. (Lessons 04–06) Define what an Effect is *for*. Explain why the dependency array is a consequence of the code rather than a configuration, what the cleanup function must accomplish, and what StrictMode's mount → unmount → mount sequence in development is testing.

A4. (Lesson 07) List three situations where an Effect is unnecessary and say what to do instead for each.

A5. (Lessons 08–09) Compare `useEffect`, `useLayoutEffect`, and `useInsertionEffect`: when each runs relative to commit/paint, and one legitimate use for each. Why is `useLayoutEffect` a risk for performance?

A6. (Lesson 10) What is a ref for? Name two things it is good for and two things it is not. Why must you avoid reading or writing `ref.current` during render, and what is the one documented exception?

A7. (Lessons 11–12) Explain what `useMemo`, `useCallback`, and `React.memo` each memoize, how they relate, and why `React.memo` on a child may do nothing when its parent passes an inline object. Why must your program be correct if React discards memoized values?

A8. (Lessons 13–14) Explain how context propagation works (who re-renders when a context value changes) and why a provider's value identity matters. Give two strategies to limit the cost.

A9. (Lessons 15–16) Why do custom hooks share *logic* but not *state*? State the Rules of Hooks and explain the mechanical reason behind the "same order every render" rule. Which modern API is exempt, and why can it be?

A10. (Lessons 17–18) Define derived state and give the three techniques to avoid storing it. Describe "colocation" and "lifting" and the heuristic for choosing between them.

A11. (Lessons 19–20) Name the categories of state in an application and the owner of each. Then explain *tearing* and how `useSyncExternalStore` prevents it; what two rules must `getSnapshot` and `subscribe` satisfy?

A12. (Lessons 21–22) Explain the difference between a Transition and an Action, and how `useTransition`, `useDeferredValue`, and `useOptimistic` differ in purpose. Why is wrapping a controlled input's `setState` in `startTransition` wrong?

Part B — Code prediction (24 points; 3 each)

For each snippet, state the observable result and justify it in 1–3 sentences. Assume React 19, StrictMode **off** unless stated, and a fresh mount.

B1.

```
function Counter() {
  const [count, setCount] = useState(0);
  function handle() {
    setCount(count + 1);
    setCount((c) => c + 2);
    setCount(count + 3);
  }
  console.log('render', count);
  return <button onClick={handle}>{count}</button>;
}
```

After one click: what is displayed and how many `render` logs follow the initial one?

B2.

```
function Child() {
  useLayoutEffect(() => { log('child layout'); return () => log('child layout cleanup'); });
  useEffect(() => { log('child passive'); });
  return null;
}
function Parent() {
  useLayoutEffect(() => { log('parent layout'); });
  useEffect(() => { log('parent passive'); });
  return <Child />;
}
```

List the `log` calls on the initial mount (no StrictMode), in order.

B3.

```
function Room({ roomId }: { roomId: string }) {
  useEffect(() => {
    log(`connect ${roomId}`);
    return () => log(`disconnect ${roomId}`);
  }, [roomId]);
  return null;
}
// <StrictMode><Room roomId="a" /></StrictMode>   → mounts
// later the parent re-renders with roomId="b"
```

List the logs from mount in **development** StrictMode, then from the change to `"b"`.

B4.

```
function App() {
  const [count, setCount] = useState(0);
  function handle() {
    setCount(count + 1);
    setTimeout(() => console.log('timeout', count), 100);
  }
  return <button onClick={handle}>{count}</button>;
}
```

Click once and wait. What is logged and what does the button show? Then click again after the first timeout completed; what is logged?

B5.

```
const Row = React.memo(function Row({ onSelect, id }: { onSelect: (id: string) => void; id:
string }) {
  console.log('Row', id);
  return <li onClick={() => onSelect(id)}>{id}</li>;
});

function List() {
  const [query, setQuery] = useState('');
  const [selected, setSelected] = useState<string | null>(null);
  const select = useCallback((id: string) => setSelected(id), []);
  return (
    <>
      <input value={query} onChange={(e) => setQuery(e.target.value)} />
      <ul>
        <Row id="a" onSelect={select} />
        <Row id="b" onSelect={(id) => setSelected(id)} />
      </ul>
    </>
  );
}
```

The user types one character. Which `Row` logs appear (ignoring the initial mount)?

B6.

```tsx
const ThemeContext = createContext('light');
function Provider({ children }: { children: React.ReactNode }) {
  const [tick, setTick] = useState(0);
  return (
    <ThemeContext value="dark">
      <button onClick={() => setTick(tick + 1)}>tick {tick}</button>
      {children}
    </ThemeContext>
  );
}
const Consumer = React.memo(function Consumer() {
  const theme = useContext(ThemeContext);
  console.log('Consumer', theme);
  return <p>{theme}</p>;
});
// <Provider><Consumer /></Provider>   (Consumer is passed as children from the root)
```

Click the button. Does `Consumer` log again? Now change the provider line to `<ThemeContext value={{ mode: 'dark' }}>` with the consumer reading `.mode` (and still `React.memo`, passed as `children`). Does it log?

B7.

```tsx
const store = { n: 0, listeners: new Set<() => void>() };
function useN() {
  return useSyncExternalStore(
    (cb) => { store.listeners.add(cb); return () => store.listeners.delete(cb); },
    () => ({ n: store.n }),
  );
}
function Show() { return <p>{useN().n}</p>; }
```

What happens when `Show` mounts?

B8.

```
async function add(prev: number, inc: number) {
  await new Promise((r) => setTimeout(r, 10));
  return prev + inc;
}
function Calc() {
  const [total, dispatch, isPending] = useActionState(add, 0);
  return (
    <button onClick={() => startTransition(() => { dispatch(1); dispatch(2); })}>
      {total} {isPending ? '…' : ''}
    </button>
  );
}
```

After one click and all work finishes, what is displayed, and what `prev` did the second action receive? Roughly how long did the work take?

Part C — Debugging challenges (30 points; 6 each)

For each: (1) state the symptom you would observe, (2) the root cause, (3) the fix with code, (4) how you would verify it (DevTools, test, or logging). Scoring: 1.5 per item.

C1. The frozen timer and the leaking subscription.

```
function Ticker({ step }: { step: number }) {
  const [n, setN] = useState(0);

  useEffect(() => {
    const id = setInterval(() => setN(n + step), 1000);
    return () => clearInterval(id);
  }, []);

  useEffect(() => {
    const onKey = (e: KeyboardEvent) => { if (e.key === 'r') setN(0); };
    window.addEventListener('keydown', onKey);
  }, []);

  return <p>{n}</p>;
}
```

Users report "the counter shows 1 forever, and after navigating away and back, pressing R behaves strangely."

C2. The flash and the stale draft.

```
function Editor({ doc }: { doc: Doc }) {
  const [title, setTitle] = useState(doc.title);
  const [wordCount, setWordCount] = useState(0);

  useEffect(() => { setWordCount(doc.body.split(/\s+/).filter(Boolean).length); },
[doc.body]);

  return (
    <>
      <input value={title} onChange={(e) => setTitle(e.target.value)} />
      <p>{wordCount} words</p>
    </>
  );
}
// parent switches `doc` when the user clicks another document in the sidebar
```

Symptoms: "After selecting another document, the title input still shows the previous document's title, and the word count briefly shows the old number."

C3. The remounting modal and the re-render storm.

```
function Page() {
  const [text, setText] = useState('');
  const [data] = useState(() => makeBigTable());

  const Modal = () => (
    <dialog open>
      <input value={text} onChange={(e) => setText(e.target.value)} autoFocus />
    </dialog>
  );

  const ctx = { text, setText };
  return (
    <TextContext value={ctx}>
      <BigTable data={data} />
      <Modal />
    </TextContext>
  );
}
```

Symptoms: "The text input loses focus after every keystroke. Also the whole table re-renders on every keystroke even though it doesn't use `text`." (`BigTable` is `React.memo`'d and does not read `TextContext`.)

C4. The wrong user.

```tsx
function Profile({ id }: { id: string }) {
  const [user, setUser] = useState<User | null>(null);
  useEffect(() => {
    fetch(`/api/users/${id}`).then((r) => r.json()).then(setUser);
  }, [id]);
  return <h1>{user?.name}</h1>;
}
```

Symptoms: "Switching quickly between profile 1 and profile 2 on a slow network sometimes ends showing profile 1 while the URL says 2. In dev the network tab shows duplicate requests on first load."

C5. The vanishing form input and the dead submit button.

```tsx
function Contact() {
  const [state, formAction] = useActionState(send, { error: '' });
  const { pending } = useFormStatus();
  return (
    <form action={formAction}>
      <input name="email" />
      <button disabled={pending}>Send</button>
      {state.error && <p>{state.error}</p>}
    </form>
  );
}
async function send(prev: { error: string }, fd: FormData) {
  const email = String(fd.get('email') ?? '');
  if (!email.includes('@')) return { error: 'Invalid email' };
  await post(email);
  return { error: '' };
}
```

Symptoms: "When the email is invalid, the error shows but the email field is cleared. The button never appears disabled while sending."

Part D — Implementation (30 points; 10 each)

Write TypeScript React code. Provide complete, compilable code for each (imports omitted is fine). Scoring is by rubric (see key).

D1. `useFetch<T>(url)`. Implement a hook returning a discriminated union `{ status: 'loading' } | { status: 'success'; data: T } | { status: 'error'; error: Error }`. Requirements: abort in-flight requests when `url` changes/unmounts; ignore aborted-request errors; never show data belonging to a *previous* URL as if it belonged to the current one (not even for one render); handle non-2xx; no `useEffect` that only sets derived state. Do not use a library.

D2. Cart with reducer and split contexts. Implement `CartProvider`, `useCart()` (state), and `useCartDispatch()` with: items `{ sku, name, priceCents, qty }`; actions `add`, `remove`, `setQty` (qty ≤ 0 removes the item), `clear`; derived `totalCents` and `count` **not stored in state**; the dispatch context value must be referentially stable; reducer must be pure and immutable; `useCart()` must throw a helpful error outside a provider.

D3. Search page. Implement `SearchPage` for a `products: readonly Product[]` prop (already in memory, ~5,000 items with an expensive row component). Requirements: a text input whose value is always responsive; filtering results by name; the committed query lives in the URL (`?q=`) with `replace` semantics after a 300 ms pause; Back/Forward must update the input and the results; results rendering must not block typing; show a subtle stale indicator while results lag. You may use the following (already written) helpers:

```
function useSearchParam(key: string): string | null;                    //
useSyncExternalStore-based
function setSearchParam(key: string, value: string | null, mode?: 'push' | 'replace'): void;
```

Part E — Architecture and reasoning (16 points; 4 each)

Answer in ≤ 200 words each. Credit is for explicit criteria and trade-offs, including at least one case where your choice would be wrong.

E1. State placement. For a logistics dashboard — shipments table (server data), selected filters (status, carrier, date range), selected shipment id for a side panel, an unsaved "add note" draft in the side panel, a "compact rows" preference, and a live count of "delayed shipments" pushed over WebSocket — decide for each datum: where it lives (local, lifted, context, external store, URL, server cache) and why.

E2. Context vs store. A teammate proposes putting the entire app state (user, cart, UI flags, live price ticks) in one React context "for simplicity". Evaluate. Propose an alternative structure, and say what evidence (measurements) would change your mind.

E3. Optimistic or not. For each, decide optimistic UI vs explicit pending UI, and justify: (a) toggling "favorite" on a product, (b) submitting a bank transfer, (c) reordering Kanban cards with server persistence, (d) posting a chat message.

E4. Refactor strategy. You inherit a 600-line `useDashboard()` hook with 20 `useState`s and 12 `useEffect`s used by eight components, with few tests. Outline an incremental plan: first three steps, safety nets, and how you decide when to stop.

Answer Key

> Attempt the assessment first. Points are shown per item; partial credit is allowed at 1-point granularity.

Part A — Key ideas (3 points each: 1 per bold idea)

A1.

1. **Each render has its own fixed** `count`: state in a render is a constant snapshot (closures in that render's handlers see that value); updating state doesn't change the variable in the running handler.
2. `setCount(count + 1)` **×3 enqueues "set to 1" three times** (all computed from the same snapshot `0`), the last wins → 1. `setCount(c => c + 1)` enqueues *updater functions* applied in order to the latest pending state → +3.
3. **The update queue lives on the component's hook state in React (the Fiber), not in the variable**; updates are batched and processed on the next render (Phase 1 / 15, Phase 2 / 22). *(Acceptable: "stored in React's internal hook queue; implementation detail"; do not require exact internal names.)*

A2. Any two technical reasons (2 points) plus the contrasting case (1 point).

- **Multiple fields change together / transitions between states must be consistent** (invalid combinations prevented via a single transition function).
- **Next state depends on previous state and an action** → logic centralized, testable as a pure function without React; `dispatch` identity stable (useful for deps/context).
- **Many handlers update the same state** or logic is complex/branchy.
- `useState` better when state is a single independent value or simple (toggle, input text); a reducer is ceremony.

A3.

1. **An Effect synchronizes the component with an external system** (network, subscription, DOM API, timer) — not an event handler or derived data.
2. **Dependencies are what the effect reads (reactive values)**; the linter reflects the code; you change the code, not the array, to change dependencies. Cleanup must **stop/undo whatever the setup started, symmetrically**, so re-running is safe (setup → cleanup → setup).
3. **StrictMode's extra mount/unmount/mount in dev checks that effects are resilient** to being torn down and re-run (e.g., future Offscreen/Activity behavior, fast refresh); bugs surface as duplicate subscriptions/requests (Phase 2 / 28).

A4. Any three, each with an alternative (1 point each if the cause *and* alternative are correct): derived data (compute during render/ `useMemo`); reset state on prop change (use `key`, or adjust during render); event-specific logic (event handler); notifying parent (call in handler / lift state); chains of effects (single event handler / derive); subscribing to external store (`useSyncExternalStore`); initializing app singletons (module scope); fetching (data library/loader, or effect with cleanup if no alternative).

A5.

1. `useEffect`: runs after commit, typically after paint (passive); for synchronization that doesn't need to block paint (subscriptions, analytics, fetch).
2. `useLayoutEffect`: runs after DOM mutations, before the browser paints; for measuring layout and mutating DOM synchronously (tooltip positioning, scroll restoration). It **blocks paint**; heavy work there delays first paint/INP. `useInsertionEffect`: runs before layout effects when DOM is about to be mutated; **for CSS-in-JS libraries to inject style rules** before layout reads; app code shouldn't use it.
3. Order: insertion → (DOM mutation) → layout → paint → passive (Phase 2 / 19).

A6.

1. **A mutable box that persists across renders and doesn't trigger re-render**; good for DOM nodes, timer ids, previous values, instance-like mutable data not used for rendering.
2. **Not for data that appears in the output** (use state) nor for synchronizing derived values.
3. **Render must be pure**; reading/writing refs in render makes output depend on hidden mutable state and breaks under multiple/discarded renders; **exception: lazy initialization** (`if (ref.current === null) ref.current = ...`).

A7.

1. `useMemo` **memoizes a computed value;** `useCallback(fn, deps)` **is** `useMemo(() => fn, deps)`**;** `React.memo` **memoizes a component's rendered output keyed by props** (shallow comparison).
2. **An inline object/function is a new reference each parent render, so** `memo` **'s shallow prop comparison sees a change**; memoize/stabilize the prop (or restructure).
3. **Memoization is a performance hint**, React may discard cached values (e.g., memory pressure/offscreen), so correctness must not depend on it (e.g., don't use it for identity-bearing values like random ids).

A8.

1. **When the provider's** `value` **changes (** `Object.is` **), all components that read that context re-render, regardless of which field they use**; intermediate components not reading it don't re-render because of it (unless their parent re-rendered).
2. **A new object literal as value each provider render makes every consumer re-render whenever the provider renders**; memoize the value or keep provider rendering rare.
3. Strategies (any two for the point): split contexts by update frequency (state vs dispatch); memoize value; keep providers low and pass `children` through; selector-based external store; `memo` on consumers' expensive children.

A9.

1. **Each call to a custom hook creates its own hook slots within the calling component**: two components calling `useCounter` get independent state; the hook is a function that composes other hooks.
2. **Rules**: call at the top level (not in conditions/loops/nested functions/after early return), only from components or custom hooks. **Mechanical reason: hook state is associated with the component instance by call order**, so changing order/count mismatches slots (Phase 2 / 17; storage layout is a teaching model).

3. `use` is exempt (may be conditional/in loops) because it reads a promise/context rather than allocating a persistent hook slot.

A10.

1. **Derived state = a value that can be computed from other state/props**; storing it creates a second source of truth.
2. **Techniques**: compute during render; `useMemo` if expensive (after measuring); key-reset or adjust-during-render when a draft must reset with identity; (also: lift the source).
3. **Colocate** state in the lowest component that needs it (limits re-render scope); **lift** to the nearest common parent when siblings need it; move down first, lift only on demand.

A11.

1. **Categories & owners**: ephemeral UI (component), form (form/draft then server), server (remote, cached), URL (address bar), shared client (lifted/context/store), derived (none).
2. **Tearing**: different components in one render pass see different values of a mutable external store when rendering is interruptible; `useSyncExternalStore` **makes React re-check the snapshot and re-render synchronously if the store changed during render**, so the committed UI is consistent.
3. **Rules**: `getSnapshot` returns an **immutable cached value** (same reference if unchanged); `subscribe` must be **stable and call the callback on change** (and unsubscribe fully); `getServerSnapshot` required for SSR.

A12.

1. **Transition = non-urgent update that can be interrupted and avoids hiding visible UI behind fallbacks; Action = function run in a transition (can be async) with pending state**; `<form action>` runs its function as an Action.
2. `useTransition` / `startTransition` **— you wrap the setter;** `useDeferredValue` **— you defer a value you receive;** `useOptimistic` **— a temporary overlay of state while an Action is pending, discarded when it completes.**
3. **A controlled input's value must update urgently** — a non-urgent update makes typed characters lag/revert; instead keep the input state urgent and transition/defer the expensive derived render.

Part B — Answers (3 points each: result 1, reasoning 1–2)

B1. Displays **3**; **one** additional render log (`render 3`). Queue: `set(0+1)=1` , then updater `1+2=3` , then `set(0+3)=3` ; result 3. Updates are batched into one render. (Common wrong answer: 6 or 4.)

B2. `child layout` , `parent layout` , `child passive` , `parent passive` . Layout effects run during commit (children before parents) and all before the passive effects, which run after paint (or flushed before the next render), also child-first. (Note: the cleanup of the layout effect logs only on re-run/unmount.)

B3. Dev StrictMode mount: `connect a` , `disconnect a` , `connect a` . Change to `"b"` : `disconnect a` , `connect b` . (Cleanup of the previous effect with the *old* closure value runs before the new setup.) If a student writes both effect runs without the simulated cleanup, they've missed the StrictMode check.

B4. After the click the button shows **1**; the timeout logs `timeout 0` because the handler closed over `count` of the render in which it was created (snapshot). After the first update, the second click's handler sees `count = 1`, sets 2, and later logs `timeout 1`.

B5. Only `Row b` logs: the typed character re-renders `List`; `Row a` receives the same `id` and the stable `select` → `memo` bails out; `Row b` receives a new inline function each render → re-renders. (Initial mount logs both.)

B6. First case: **no** log. The context value is the constant string `"dark"`; clicking re-renders `Provider`, but `children` is the same element reference created by the root, so React bails out on `Consumer`; and context didn't change anyway. Second case (`value={{ mode: 'dark' }}`): **yes** — a new object each provider render changes the context value, so consumers re-render despite `memo` (memo doesn't block context-driven updates). Fix: `useMemo` the value or hoist a constant.

B7. Infinite loop/warning: `getSnapshot` returns a new object each call, so `Object.is` never matches; React warns "The result of getSnapshot should be cached to avoid an infinite loop" and then throws "Maximum update depth exceeded". Fix: return the primitive `store.n`. Also `subscribe` is inline (re-subscribes each render) and there's no `getServerSnapshot` for SSR.

B8. Displays **3**. Actions are queued and run sequentially: first gets `prev=0` (→1), second gets `prev=1` (→3). Total ≈ **20 ms** (two sequential 10 ms waits), not 10. While running, `isPending` is true (shown as "…"). If the answer says the second received `0`, it forgot the queue semantic.

Part C — Debugging (6 points: 1.5 per item)

C1.

- *Symptom:* counter shows 1 forever (stuck), listener leak.
- *Root causes:* (1) the interval effect has `[]` deps but reads `n` and `step`: both captured from the first render (`n=0`, initial `step`) → `setN(0 + step)` forever (stale closure, P3). (2) The keydown effect has **no cleanup**; every mount (including StrictMode's second and later remounts) adds a listener that is never removed; after navigating away, the stale listeners call `setN` on unmounted components/duplicate handlers.
- *Fix:*

```
useEffect(() => {
  const id = setInterval(() => setN((c) => c + step), 1000);
  return () => clearInterval(id);
}, [step]);

useEffect(() => {
  const onKey = (e: KeyboardEvent) => { if (e.key === 'r') setN(0); };
  window.addEventListener('keydown', onKey);
  return () => window.removeEventListener('keydown', onKey);
}, []);
```

(If `step` changes shouldn't restart the timer, use `useEffectEvent` (stable since React 19.2) for the increment.)

- *Verify:* with StrictMode on, add/remove listener counts via DevTools "Event Listeners" panel or a temporary log; test with fake timers that three ticks yield `3*step`.
- Points: symptom/cause (1.5), cleanup bug identified (1.5), correct fix incl. functional update + cleanup (1.5), verification (1.5).

C2.

- *Symptom:* title from the previous document; word count flashes old.
- *Root causes:* (1) `useState(doc.title)` mirrors a prop: initial value only; the component instance is reused when `doc` changes (same position/type), so draft state persists. (2) `wordCount` is derived state stored via an effect → one render with stale count.
- *Fix:*

```
function Editor({ doc }: { doc: Doc }) {
  const [title, setTitle] = useState(doc.title);
  const wordCount = doc.body.split(/\s+/).filter(Boolean).length; // derive (useMemo only if
measured)
  ...
}
// parent: <Editor key={doc.id} doc={doc} />   // identity reset for the draft
```

- *Verify:* switch documents in dev; React DevTools shows `State` reset on key change; Profiler shows one commit per switch; test: render with doc A, edit title, rerender with doc B, assert title is B's.
- Points: both causes (3), fix with key + derive (1.5), verification (1.5).

C3.

- *Symptom 1 (focus loss):* `Modal` is defined inside `Page`; every `Page` render creates a new component type, so React unmounts/remounts the dialog subtree including the input; focus lost (also `autoFocus` refires).
- *Symptom 2 (table re-renders):* `ctx` is a new object each render → context value changes each render; but `BigTable` doesn't read the context. The real cause of the table re-render: `<BigTable data={data} />` is inside `Page`, which re-renders on each keystroke (state `text` lives in `Page`); `React.memo` should prevent `BigTable` re-rendering **if props are stable** — `data` is stable (lazy init), so `BigTable` should bail out. If it still re-renders, check whether `BigTable` internally consumes the context or receives other unstable props. So the grading accepts: (a) identifying that `memo` should hold and verifying via Profiler "why did this render", or (b) noting `Page` re-render re-creates `Modal`/context; the table re-render claim in the prompt must be verified with the Profiler rather than assumed. *The statement in the prompt is a trap: the primary bug is the inline component; the context value instability affects only consumers (here the modal's remounted subtree).* Full marks for *not* blindly "fixing" the table claim and for proposing measurement.
- *Fix:*

```
function TextModal() {                      // module scope; owns its own text state
  const [text, setText] = useState('');
  return <dialog open><input value={text} onChange={(e) => setText(e.target.value)} autoFocus
/></dialog>;
}
function Page() {
  const [data] = useState(() => makeBigTable());
  return (<><BigTable data={data} /><TextModal /></>);
}
```

State colocated in the modal: typing no longer re-renders `Page` at all. If `text` must be shared, use a memoized context value (`useMemo`) and split state/dispatch.

- *Verify:* typing keeps focus; Profiler shows only `TextModal` commits per keystroke.
- Points: remount cause (1.5), identity of context/memo analysis with measurement (1.5), fix with colocation (1.5), verification (1.5).

C4.

- *Symptom:* wrong profile shown after rapid switching; duplicate dev requests.
- *Root cause:* no cleanup → responses may resolve out of order; the slower earlier response calls `setUser` last (race condition). Duplicates in dev are StrictMode's mount/unmount/mount (which reveals the missing cleanup). Also `user` from the previous id is displayed while the next loads, and errors/non-2xx aren't handled.
- *Fix:* `AbortController` and ignore aborted errors; or `ignore` flag; store `id` with the result to avoid showing a different id's data:

```
useEffect(() => {
  const ac = new AbortController();
  fetch(`/api/users/${id}`, { signal: ac.signal })
    .then((r) => { if (!r.ok) throw new Error(String(r.status)); return r.json(); })
    .then(setUser)
    .catch((e) => { if (!ac.signal.aborted) setError(e); });
  return () => ac.abort();
}, [id]);
```

(Preferred in production: a data library.)

- *Verify:* DevTools Network throttling "Slow 3G", rapid id switching; first request canceled in the Network tab; test with MSW delaying request 1 longer than request 2.
- Points: race cause (1.5), StrictMode explanation (1.5), fix (1.5), verification (1.5).

C5.

- *Symptom 1:* field cleared on invalid email: React resets uncontrolled form fields after the Action completes; the action returns only `{error}` , so there's nothing to restore. *Fix:* return submitted values and bind `defaultValue` (or use a controlled input) (Verified on React 19.3.1: after a function Action completes, React resets the form's uncontrolled fields to their `defaultValue` ; with no

`defaultValue` the typed text is cleared, with `defaultValue={state.email}` the returned value is shown.)

- *Symptom 2:* `useFormStatus()` is called in the same component that renders the `<form>`; it only reads the status of a *parent* form, so `pending` is always `false`. *Fix:* move into a child `SubmitButton`, or use the third value `isPending` from `useActionState`.

```
function SubmitButton() { const { pending } = useFormStatus(); return <button disabled=
{pending}>Send</button>; }

function Contact() {
  const [state, formAction] = useActionState(send, { error: '', email: '' });
  return (
    <form action={formAction}>
      <input name="email" defaultValue={state.email} aria-invalid={Boolean(state.error)} />
      <SubmitButton />
      {state.error && <p role="alert">{state.error}</p>}
    </form>
  );
}
// send returns { error, email } in both branches
```

- Also acceptable observation: `post` errors thrown inside the action go to an Error Boundary and cancel queued actions; return error state instead.
- *Verify:* test with user-event: type invalid email, submit, assert the input still contains it and error is visible; assert the button is disabled while `post` is pending (fake a delayed MSW response).
- Points: reset (1.5), hook placement (1.5), fixes (1.5), verification (1.5).

Part D — Implementation key (10 points each)

D1. `useFetch<T>` — rubric

Criterion	Pts
Discriminated union type; no impossible states	2
`AbortController` created in the effect, aborted in cleanup; aborted errors ignored	2
Result tied to its `url`; no stale-URL data for any render (derive `loading` when `settled.url !== url`)	3
Non-2xx handled as error; JSON typed; unknown error normalized to `Error`	1
No derived-state effect; deps `[url]` honest	2

Reference solution:

```ts
type FetchState<T> =
  | { status: 'loading' }
  | { status: 'success'; data: T }
  | { status: 'error'; error: Error };

type Settled<T> = { url: string } & (
  | { status: 'success'; data: T }
  | { status: 'error'; error: Error }
);

export function useFetch<T>(url: string): FetchState<T> {
  const [settled, setSettled] = useState<Settled<T> | null>(null);

  useEffect(() => {
    const ac = new AbortController();
    fetch(url, { signal: ac.signal })
      .then((r) => {
        if (!r.ok) throw new Error(`HTTP ${r.status}`);
        return r.json() as Promise<T>;
      })
      .then((data) => setSettled({ url, status: 'success', data }))
      .catch((e: unknown) => {
        if (ac.signal.aborted) return;
        setSettled({ url, status: 'error', error: e instanceof Error ? e : new
Error(String(e)) });
      });
    return () => ac.abort();
  }, [url]);

  if (settled === null || settled.url !== url) return { status: 'loading' };
  return settled.status === 'success'
    ? { status: 'success', data: settled.data }
    : { status: 'error', error: settled.error };
}
```

Why this design: loading is *derived* (result belongs to a different URL → loading) so the first render after a `url` change is already correct; no `setState({status:'loading'})` inside the effect is needed. StrictMode: the first request is aborted by the simulated unmount, the second completes. Trade-offs: no caching/dedupe/retry — use a data library in production; the module doesn't clear stale `settled` (fine; overwritten). Edge case: same URL requested again after an error doesn't refetch unless `url` changes — add a `reloadKey` param if needed.

D2. Cart — rubric

Criterion	Pts
Typed state/actions (discriminated union), exhaustive reducer, pure/immutable	3

Criterion	Pts
`setQty` ≤ 0 removes; `add` merges existing sku	1
Derived `totalCents`, `count` computed (not stored); money as integer cents	2
Two contexts (state/dispatch); dispatch stable (from `useReducer`); state value not recreated needlessly	2
Hooks throw outside provider (`null` default)	1
`children` pass-through so provider re-render doesn't re-render the whole subtree	1

Reference:

```tsx
interface CartItem { sku: string; name: string; priceCents: number; qty: number }
type CartAction =
  | { type: 'add'; item: Omit<CartItem, 'qty'> }
  | { type: 'remove'; sku: string }
  | { type: 'setQty'; sku: string; qty: number }
  | { type: 'clear' };

function cartReducer(items: CartItem[], a: CartAction): CartItem[] {
  switch (a.type) {
    case 'add': {
      const existing = items.find((i) => i.sku === a.item.sku);
      return existing
        ? items.map((i) => (i.sku === a.item.sku ? { ...i, qty: i.qty + 1 } : i))
        : [...items, { ...a.item, qty: 1 }];
    }
    case 'remove':
      return items.filter((i) => i.sku !== a.sku);
    case 'setQty':
      return a.qty <= 0
        ? items.filter((i) => i.sku !== a.sku)
        : items.map((i) => (i.sku === a.sku ? { ...i, qty: a.qty } : i));
    case 'clear':
      return [];
  }
}

const CartStateContext = createContext<CartItem[] | null>(null);
const CartDispatchContext = createContext<React.Dispatch<CartAction> | null>(null);

export function CartProvider({ children }: { children: React.ReactNode }) {
  const [items, dispatch] = useReducer(cartReducer, []);
  return (
    <CartDispatchContext value={dispatch}>
      <CartStateContext value={items}>{children}</CartStateContext>
    </CartDispatchContext>
  );
}

export function useCart() {
  const items = useContext(CartStateContext);
  if (items === null) throw new Error('useCart must be used inside <CartProvider>');
  const totalCents = items.reduce((s, i) => s + i.priceCents * i.qty, 0);
  const count = items.reduce((n, i) => n + i.qty, 0);
  return { items, totalCents, count };
}
```

```
export function useCartDispatch() {
  const dispatch = useContext(CartDispatchContext);
  if (dispatch === null) throw new Error('useCartDispatch must be used inside
<CartProvider>');
  return dispatch;
}
```

Notes: `items` is replaced only by the reducer, so the state context value changes only when the cart changes (no `useMemo` needed because the value *is* the state, not a wrapper object). Components that only dispatch (add buttons) read only the dispatch context and don't re-render on cart changes. `useCart` returns a new small object each call, which is fine unless used as a dependency (document it, or return derived values separately/with `useMemo`). In React 18, use `<Context.Provider>`.

D3. Search page — rubric

Criterion	Pts
Urgent local `draft` state controls the input (never transitioned)	2
Committed query read from URL via `useSearchParam`; written with `replace` after 300 ms with effect cleanup (clearTimeout)	2
Back/Forward sync: draft adjusted when committed value changes externally, without an effect loop (adjust-during-render or key) and without writing back stale draft	2
Results use `useDeferredValue` on the *draft or committed* value + `memo`'d list with stable props; stale indicator derived (`draft !== deferred`)	2
Filtering derived in render (`useMemo` acceptable), no stored filtered state; accessibility basics (label, `aria-busy`)	2

Reference:

```tsx
const ResultList = memo(function ResultList({ products, query }: { products: readonly
Product[]; query: string }) {
  const q = query.trim().toLowerCase();
  const visible = useMemo(
    () => (q ? products.filter((p) => p.name.toLowerCase().includes(q)) : products),
    [products, q],
  );
  return <ul>{visible.map((p) => <ProductRow key={p.id} product={p} />)}</ul>;
});

export function SearchPage({ products }: { products: readonly Product[] }) {
  const committed = useSearchParam('q') ?? '';
  const [draft, setDraft] = useState(committed);

  // URL changed from outside (Back/Forward, link): adopt it during render
  const [prevCommitted, setPrevCommitted] = useState(committed);
  if (committed !== prevCommitted) {
    setPrevCommitted(committed);
    setDraft(committed);
  }

  // commit draft -> URL after 300 ms of silence; replace so typing doesn't spam history
  useEffect(() => {
    if (draft === committed) return;
    const id = setTimeout(() => setSearchParam('q', draft, 'replace'), 300);
    return () => clearTimeout(id);
  }, [draft, committed]);

  const deferred = useDeferredValue(draft);
  const isStale = deferred !== draft;

  return (
    <>
      <label>
        Search products
        <input value={draft} onChange={(e) => setDraft(e.target.value)} />
      </label>
      <div aria-busy={isStale} style={{ opacity: isStale ? 0.6 : 1 }}>
        <ResultList products={products} query={deferred} />
      </div>
    </>
  );
}
```

Design notes (what a strong answer says): the URL holds the committed value (shareable, Back-friendly); the draft keeps typing independent of URL writes; deferral keeps typing responsive while the heavy list

renders in the background and is interruptible; `memo` + stable `products` reference make the urgent render cheap; `useDeferredValue(draft)` is used rather than the URL value because the URL lags by 300 ms (using the committed value would make results update only after the debounce — acceptable alternative: filter by `committed`, but then the "stale indicator" semantic is different). Pitfalls: pushing to history per keystroke; using `startTransition` around `setDraft`; passing the urgent `draft` to `ResultList`; omitting adjust-during-render so Back is overridden by the debounce effect (the effect would write the stale draft back after 300 ms).

Part E — Reasoning key (4 points each: ≈1 for decisions, 1.5 for criteria, 1.5 for trade-offs/counter-case)

E1. Example answer (decisions must have reasons):

- Shipments table: **server state** (cache layer keyed by filters); not copied into state.
- Filters: **URL state** (shareable dashboard views), parsed/validated at the boundary; page reset when filters change.
- Selected shipment id: **URL** (`/shipments/:id` or `?selected=`) — deep link and Back-friendly; the side panel data is a server-state query keyed by id.
- Add-note draft: **local/form state in the side panel**, keyed by shipment id (reset with `key`), warn on unsaved navigation; not global.
- Compact rows: **persisted client preference** — small store/context with localStorage and `useSyncExternalStore` / `storage` event; no URL (personal, not part of the shareable view).
- Delayed-shipments live count: **external store/cache updated by WebSocket**, header subscribes via selector; or write into the query cache. Alternative: refetch on interval if real-time isn't required. Counter-case: if dashboards are never shared and filters change dozens of times per second (e.g., slider scrubbing), keep them local and commit to URL on release; if the server pushes shipment updates, the cache must be updated by the push handler, not duplicated in a store.

E2. Evaluation: One context re-renders every consumer on any change (user, cart, flags, **ticks every few hundred ms**), so price ticks would re-render the entire app; the value identity issue; testing coupling; SSR per-request concerns. Alternative: split by domain and frequency — auth/session (context over cached server state), cart (reducer + split contexts, or server cart through the data layer), UI flags colocated, live prices in an external store with per-symbol selectors. Evidence that would change my mind: Profiler shows negligible re-render cost (small tree, low-frequency updates) and the team values simplicity; or the compiler/memoization makes the cost irrelevant — measure commits per interaction and INP before deciding. Credit for acknowledging "context is fine when updates are rare and the consumer set small".

E3. (a) Favorite: **optimistic** — cheap, reversible, almost always succeeds; toast on failure; server idempotent set-state. (b) Bank transfer: **explicit pending**, no optimistic success; high stakes, irreversible, may need confirmation/2FA; idempotency key; optionally show a "pending" transaction row clearly labeled. (c) Kanban reorder: **optimistic** with a reducer overlay and server version/ordering ops; handle rapid successive moves and out-of-order responses; revert on failure with message. (d) Chat message: **optimistic with "sending…" state** and retry/failed state (message appears immediately with pending status; client-generated id so key stays stable); don't pretend delivery. Criteria: reversibility, failure likelihood, cost of wrong display, ability to reconcile, idempotency.

E4. Example plan: (1) *Safety net first*: characterization tests at the component level for key flows, Profiler baseline, enable lint rules as errors; (2) *Inventory*: classify each state/effect (server, URL, derived, UI, external sync) — find derived-state effects and event-in-effect patterns; (3) *Delete before restructure*: remove derived state/effect chains, move events into handlers (low risk, high payoff); (4) extract pure logic into tested functions; (5) move server data to a cache/data layer and URL-appropriate state to the URL; (6) split into narrow hooks/components by concern, migrating consumers one at a time behind a compatibility facade; (7) measure; only then memoize. Stop when: the hook has one concern, no lint suppressions remain, consumers read only what they use, render counts per interaction hit targets, and new features no longer require touching the legacy hook. Credit for incremental merges and explicit risks (behavior drift, hidden dependencies between effects).

Scoring and Promotion

Rubric by category

Explanation (36 pts). For each of A1–A12, award 1 point per key idea correctly stated in the student's own words. Do not award points for restating terms without a causal link ("effects run after render" without *why/what for* earns nothing). Deduct *no* points for omitted exact internal names; do deduct for confident false claims (e.g., "refs trigger re-renders", "`useMemo` guarantees stability").

Score	Interpretation
31–36	Can teach the model; ready to mentor
26–30	Solid (pass threshold ≥ 26)
18–25	Gaps in causal reasoning; revisit lessons indicated in the question headers
< 18	Re-study Phase 3 before proceeding

Reasoning (40 pts). B (24) and E (16). For B: 1 point correct observable result; 2 points for the justification (snapshot, queue semantics, identity, effect ordering, action queue). For E: points for stated criteria and trade-offs, including a counter-case; a "right" decision with no criteria earns at most 2/4.

Debugging (30 pts). Each of C1–C5: 1.5 symptom/cause identification, 1.5 root-cause mechanism, 1.5 correct fix, 1.5 verification method. A fix that suppresses the linter or masks the symptom (e.g., adding a `ref` to stop a loop, disabling StrictMode) earns 0 for the fix criterion.

Implementation (30 pts). Rubrics under D1–D3. Code must type-check conceptually; syntax slips are tolerated, semantic errors (mutation of state, missing cleanup, derived-state effects) are scored down per rubric. Reasonable alternative designs that meet the stated requirements and rubric criteria receive full credit.

Pass thresholds

- **Overall ≥ 80% (≥ 109 / 136).**

- **Each category ≥ 70%:** Explanation ≥ 26 / 36, Reasoning ≥ 28 / 40, Debugging ≥ 21 / 30, Implementation ≥ 21 / 30.

Failing a single category fails the assessment even if the total is above 80%; remediate that category and retake **only the failed parts** with new variants (instructors: swap data/scenarios; do not reuse the same snippets).

Remediation map

Weak area	Revisit
Snapshots, batching, functional updates (A1, B1, B4)	01, 02, Phase 1 / 14–15
Reducers (A2, D2)	03
Effects: deps, cleanup, StrictMode (A3, B2, B3, C1, C4, D1)	04, 05, 06, 07
Layout/insertion effects (A5, B2)	08, 09
Refs (A6)	10
Memoization (A7, B5)	11, 12
Context (A8, B6, C3, D2, E2)	13, 14
Custom hooks and rules (A9)	15, 16
Derived state, colocation (A10, C2)	17, 18
State categories, URL/server state (A11, E1, D3)	19
External stores (A11, B7)	20
Forms and Actions (C5, B8)	21, 22
Modern hooks, transitions, optimistic UI (A12, B8, D3, E3)	22
Anti-pattern recognition (C-series, E4)	23

Promotion criteria

You may proceed to **Phase 4 — React architecture** (Phase 4 / 01) when **all** hold:

1. You meet the pass thresholds above.
2. You can, without notes, produce a correct explanation of: snapshot/queue semantics; effect synchronization with cleanup; why derived state shouldn't be stored; how context propagation costs arise; what tearing is and how `useSyncExternalStore` addresses it; the difference between Transition, deferral, and optimistic UI.
3. You have completed the Phase 3 projects: **Project 2 — Task manager (state ownership)** and **Project 4 — Complex form system** (see `projects/`), each with reviewed state-ownership documentation (which datum has which owner) and no lint suppressions of `react-hooks/exhaustive-deps` without a written justification.
4. In a 30-minute live review of an unfamiliar component with ≥ 5 hooks, you can identify at least four problems from 23 and propose fixes with their trade-offs.

Conditional promotion (80–84% overall with all categories ≥ 70%): proceed, but schedule a re-read of the remediation-map lessons for your two lowest-scoring questions before Phase 4 lesson 15 (State ownership), which assumes fluent handling of this material.

Re-study required: any category < 70%, or overall < 80%.

> Items that depend on fast-moving APIs (`useEffectEvent` , React Compiler, framework Server Functions) are assessed only for conceptual understanding and the habit of verifying against current documentation, not for memorized API details.